

hw3-1

May 20, 2026

1 CS 229 - Homework 3: Emoji Flow

Submit **PDF** of completed notebook on Canvas.

Maximum points: 15

Enter your information below:

(full) Name: Vedant Deepak Borkute Student ID Number: 862552981

By submitting this notebook, I assert that the work below is my own work. Except where explicitly cited, none of the portions of this notebook are duplicated from anyone else's work.

1.1 Overview

In this assignment you will implement a **flow matching generative model** from scratch. You'll train a small Vision Transformer to generate 32×32 emoji images by:

1. Understanding the forward noising process: linear interpolation between noise and data
2. Implementing the training loss function (weighted x-prediction)
3. Implementing an Euler ODE sampler to generate new images from noise
4. Visualizing the full pipeline: noising, denoising, and generation

Flow matching learns to reverse a simple process: given a clean image x and noise ε , define a straight-line path $z_t = t \cdot x + (1 - t) \cdot \varepsilon$. A neural network learns to predict x from z_t , and at generation time we integrate from noise back to data.

Complete all parts marked **TODO** and ensure all test cells produce the expected output.

1.2 Setup

Run these cells to load data and model. **Do not modify.**

```
[1]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[2]: %matplotlib inline
import math
import torch
import torch.nn as nn
```

```
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from torchvision.utils import make_grid
import matplotlib.pyplot as plt
import numpy as np
```

```
[3]: device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
device = torch.device('mps') if torch.backends.mps.is_available() else device
print(f'Using device: {device}')

torch.manual_seed(229)
```

Using device: cuda

```
[3]: <torch._C.Generator at 0x7a2948fad4b0>
```

1.3 Load Data

Download hw3_data_dedupe.pt from Canvas and place it in the same directory as this notebook.

The dataset contains ~1,400 Google Noto Color Emoji rendered as 32x32 RGB images, normalized to $[-1, 1]$. This is a deduplicated set — skin-tone variants and gender-combo sequences have been removed to improve diversity. We split a small validation set for monitoring overfitting.

```
[4]: data = torch.load('/content/drive/My Drive/hw3_data_dedupe.pt',
    ↪weights_only=False)
all_images = data['images']    # (N, 3, 32, 32) float32, range [-1, 1]
names = data['names']         # list of N emoji name strings

# Train/val split (fixed seed for reproducibility)
torch.manual_seed(229)
perm = torch.randperm(len(all_images))
n_val = len(all_images) // 10 # ~10% validation
images = all_images[perm[n_val:]]
val_images = all_images[perm[:n_val]]

N = len(images)
print(f'Total: {len(all_images)} emoji, shape {all_images.shape}')
print(f'Train: {N}, Val: {len(val_images)}')
print(f'Pixel range: [{all_images.min():.2f}, {all_images.max():.2f}']')
print(f'\nSample names: {names[:6]}')
```

Total: 1447 emoji, shape torch.Size([1447, 3, 32, 32])

Train: 1303, Val: 144

Pixel range: [-1.00, 1.00]

Sample names: ['NUMBER SIGN', 'NUMBER SIGN + COMBINING ENCLOSING KEYCAP',
'ASTERISK', 'ASTERISK + COMBINING ENCLOSING KEYCAP', 'DIGIT ZERO', 'DIGIT ZERO +

COMBINING ENCLOSING KEYCAP']

```
[5]: # Show a grid of sample emoji - pick a representative random subset
def show_grid(imgs, nrow=8, title=None, figsize=None):
    """Display images as a grid. imgs: (N, 3, H, W) in [-1, 1]."""
    imgs = imgs.detach().cpu().clamp(-1, 1)
    grid = make_grid((imgs + 1) / 2, nrow=nrow, padding=1, pad_value=1)
    if figsize is None:
        ncol = min(nrow, len(imgs))
        nrows = (len(imgs) + ncol - 1) // ncol
        figsize = (ncol * 1.5, nrows * 1.5 + 0.5)
    plt.figure(figsize=figsize)
    plt.imshow(grid.permute(1, 2, 0).numpy())
    if title:
        plt.title(title, fontsize=14)
    plt.axis('off')
    plt.tight_layout()
    plt.show()

torch.manual_seed(229)
show_grid(images[torch.randperm(N)[:64]], nrow=8, title=f'Sample emoji ({N}
↳total)')
```

1.4 The Model: Vision Transformer (ViT)

We use a small Vision Transformer that takes a noisy image z_t and predicts the clean image $\hat{x}$. Key design choices:

- **Patch size 8×8** $\rightarrow$ 16 patches per image, each of dimension $8 \times 8 \times 3 = 192$
- **No time conditioning:** the network receives *only* z_t — no timestep input. This works because at $D = 3,072$ dimensions, the norm of z_t is strongly informative of t (concentration of measure). See [Sahraee-Ardakan et al. \(2026\)](#), “The Geometry of Noise.”
- **x-prediction:** the output is $\hat{x}$, the predicted clean image

The architecture is provided. **Do not modify.**

```
[6]: class TransformerBlock(nn.Module):
    """Pre-norm Transformer block."""
    def __init__(self, dim, num_heads, mlp_ratio=4.0):
        super().__init__()
        self.norm1 = nn.LayerNorm(dim)
        self.attn = nn.MultiheadAttention(dim, num_heads, batch_first=True)
        self.norm2 = nn.LayerNorm(dim)
        self.mlp = nn.Sequential(
            nn.Linear(dim, int(dim * mlp_ratio)),
            nn.GELU(),
            nn.Linear(int(dim * mlp_ratio), dim),
        )

    def forward(self, x):
        h = self.norm1(x)
        h, _ = self.attn(h, h, h)
        x = x + h
        x = x + self.mlp(self.norm2(x))
        return x

class EmojiViT(nn.Module):
    """Vision Transformer for emoji generation. No time conditioning."""
    def __init__(self, img_size=32, patch_size=8, in_channels=3,
                  embed_dim=256, depth=6, num_heads=4, mlp_ratio=4.0):
        super().__init__()
        self.patch_size = patch_size
        self.num_patches = (img_size // patch_size) ** 2 # 16
        patch_dim = in_channels * patch_size ** 2 # 192

        self.patch_embed = nn.Linear(patch_dim, embed_dim)
        self.pos_embed = nn.Parameter(torch.randn(1, self.num_patches,
        ↪ embed_dim) * 0.02)
        self.blocks = nn.Sequential(*[
            TransformerBlock(embed_dim, num_heads, mlp_ratio) for _ in
        ↪ range(depth)
        ])
        self.norm = nn.LayerNorm(embed_dim)
        self.head = nn.Linear(embed_dim, patch_dim)

    def patchify(self, x):
        """(B, C, H, W) -> (B, num_patches, patch_dim)"""
        B, C, H, W = x.shape
        p = self.patch_size
        x = x.reshape(B, C, H // p, p, W // p, p)
        x = x.permute(0, 2, 4, 1, 3, 5) # (B, H//p, W//p, C, p, p)
        x = x.reshape(B, self.num_patches, -1) # (B, num_patches, C*p*p)
```

```

    return x

def unpatchify(self, x):
    """(B, num_patches, patch_dim) -> (B, C, H, W)"""
    B = x.shape[0]
    p = self.patch_size
    h = w = int(self.num_patches ** 0.5)
    x = x.reshape(B, h, w, 3, p, p)
    x = x.permute(0, 3, 1, 4, 2, 5)          # (B, 3, h, p, w, p)
    x = x.reshape(B, 3, h * p, w * p)
    return x

def forward(self, x):
    x = self.patchify(x)          # (B, 16, 192)
    x = self.patch_embed(x)       # (B, 16, 256)
    x = x + self.pos_embed
    x = self.blocks(x)           # (B, 16, 256)
    x = self.norm(x)
    x = self.head(x)             # (B, 16, 192)
    x = self.unpatchify(x)        # (B, 3, 32, 32)
    return x

```

```

[7]: # Verify the model
model = EmojiViT().to(device)
n_params = sum(p.numel() for p in model.parameters())
print(f'EmojiViT: {n_params:,} parameters')

test_in = torch.randn(2, 3, 32, 32, device=device)
test_out = model(test_in)
print(f'Input: {test_in.shape}')
print(f'Output: {test_out.shape}')
assert test_out.shape == test_in.shape, 'Output shape must match input shape!'
print('Shape check passed.')

```

```

EmojiViT: 4,841,920 parameters
Input: torch.Size([2, 3, 32, 32])
Output: torch.Size([2, 3, 32, 32])
Shape check passed.

```

1.5 Part 0: The Noising Process [2 points]

In flow matching, we define a straight-line path between noise $\varepsilon \sim \mathcal{N}(0, I)$ and a clean image x :

$$z_t = t \cdot x + (1 - t) \cdot \varepsilon$$

At $t = 0$, we have pure noise. At $t = 1$, we recover the clean image.

[2 points] Implement `add_noise` below — it samples fresh noise and returns the noisy image. You will reuse this function in the training loss (Part 1).

```
[8]: def add_noise(x, t):
    """Create noisy image by linear interpolation with fresh noise.

    Args:
        x: clean images, shape (B, C, H, W) or (C, H, W)
        t: time value(s), scalar or shape (B, 1, 1, 1) - 0 = pure noise, 1 =
        ↪ clean

    Returns:
        z_t: noisy images, same shape as x
    """
    # TODO [2 points]: Sample noise  $\epsilon \sim N(0, I)$  and return  $t * x + (1 - t) * \epsilon$ 
    ↪  $\epsilon$ 
    noise = torch.randn_like(x)
    return t * x + (1 - t) * noise
```

1.5.1 Viz 0: Check your `add_noise`

The cell below calls your function at several noise levels. You should see a smooth interpolation from pure noise (left) to clean image (right).

```
[9]: # Viz 0 - visualize the noising process (uses your add_noise function)
t_values = [0.0, 0.25, 0.5, 0.75, 0.99]

torch.manual_seed(229)
viz_indices = torch.randperm(N)[:4].tolist()

fig, axes = plt.subplots(len(viz_indices), len(t_values), figsize=(12, 10))

for row, idx in enumerate(viz_indices):
    x = images[idx]

    for col, t_val in enumerate(t_values):
        z = add_noise(x, t_val)
        img = ((z.permute(1, 2, 0).numpy() + 1) / 2).clip(0, 1)
        axes[row, col].imshow(img)
        axes[row, col].axis('off')
        if row == 0:
            axes[0, col].set_title(f't = {t_val}', fontsize=13)

fig.suptitle('Viz 0: The Noising Process (noise ↪ clean)', fontsize=14)
plt.tight_layout()
plt.show()
```



1.6 Part 1: Train the Denoiser [7 points]

1.6.1 1a. The Training Loss Function [2 points]

The network predicts clean data: $\hat{x} = f_{\theta}(z_t)$. The training loss is the **weighted x-prediction loss**:

$$\mathcal{L} = \mathbb{E}_{t,x,\varepsilon} \left\| \frac{\hat{x} - x}{1 - t} \right\|^2$$

Why the $1/(1-t)$ weight? This is algebraically identical to the velocity-prediction loss $\|v_{\theta} - v\|^2$, where $v = x - \varepsilon$. Since $\hat{x} = z_t + (1 - t) \cdot v_{\theta}$, the x-prediction error scaled by $1/(1 - t)$ equals the v-prediction error. This was shown in lecture and follows from [Li & He \(2025\)](#).

Time sampling: logit-normal. Rather than sampling t uniformly, we use a **logit-normal** distribution that concentrates training on mid-range noise levels (where denoising is hardest). Sample $t = \text{sigmoid}(\mu + \sigma \cdot z)$ with $z \sim \mathcal{N}(0, 1)$, $\mu = -0.8$, $\sigma = 0.8$. This follows [Li & He \(2025\)](#).

Implement `compute_loss` below. Your function should: 1. Sample t via logit-normal (use $\mu = -0.8$, $\sigma = 0.8$), reshaped for broadcasting 2. Compute the noisy image z_t using your `add_noise` function 3. Get the model's prediction $\hat{x} = \text{model}(z_t)$ 4. Return the weighted MSE loss

Note: this loss is stochastic - even for the same x , each call gives a different estimate due to random t sampling and random noise. The training loop averages over many such estimates.

```
[10]: def compute_loss(model, x):
    """Compute the weighted x-prediction loss for flow matching.

    Args:
        model: EmojiViT that maps  $z_t \rightarrow \hat{x}$  (no time input)
        x: (B, 3, 32, 32) batch of clean images

    Returns:
        loss: scalar tensor
    """
    B = x.shape[0]

    # TODO [2 points]: Implement the 4 steps described above.
    #
    # Step 1: Sample  $t$  via logit-normal, reshaped to (B, 1, 1, 1) for
    ↪broadcasting
    #  $t = \text{torch.distributions.LogNormal}(-0.8, 0.8).sample()$ 
    B = x.shape[0]
    t = torch.sigmoid(-0.8+0.8*(torch.randn(B, device = x.device))) #creates it
    ↪on cpu by default
    # print(t.shape)
    t = t.reshape(B, 1, 1, 1)
    # Step 2:  $z_t = \text{add\_noise}(x, t)$ 
    z_t = add_noise(x, t)
    # Step 3:  $\hat{x} = \text{model}(z_t)$ 
    x_hat = model(z_t)
    # Step 4: loss = mean of  $((\hat{x} - x) / (1 - t))^2$ 
    loss = torch.mean(((x_hat - x) / ((1-t)))) ** 2
    return loss
```

```
[11]: # Test compute_loss
test_x = images[:4].to(device)
test_loss = compute_loss(model, test_x)
print(f'Loss on 4 images: {test_loss.item():.4f}')
print(f'Is scalar: {test_loss.dim() == 0}')
print(f'Is finite: {torch.isfinite(test_loss).item()}')

# Check gradient flows
test_loss.backward()
```

```

grad_norm = sum(p.grad.norm().item() for p in model.parameters() if p.grad is_
↳not None)
print(f'Gradient norm: {grad_norm:.4f} (should be > 0)')
model.zero_grad()
print('\nAll checks passed!')

```

```

Loss on 4 images: 1.8921
Is scalar: True
Is finite: True
Gradient norm: 32.9173 (should be > 0)

```

All checks passed!

1.6.2 1b. Train the Model [1 point]

Run the training loop below. This uses your `compute_loss` function — if it's implemented correctly, the loss should decrease steadily.

Tip: Use a small number of epochs (e.g., 20-50) while debugging your `compute_loss`. Once everything works, train for ~500 epochs to get reasonable generation quality. Training much longer tends to overfit.

```

[12]: # Training loop - should work without modification
epochs = 500
batch_size = 128
lr = 1e-3

dataset = TensorDataset(images)
loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

model = EmojiViT().to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=0.01)

# Validation data on device for periodic evaluation
val_batch = val_images.to(device)

print(f'Training for {epochs} epochs, {len(loader)} batches/epoch')
print(f'Model: {sum(p.numel() for p in model.parameters()):,} parameters')
print()

train_losses = []
val_losses = []
for epoch in range(epochs):
    model.train()
    epoch_loss = 0
    for (batch,) in loader:
        batch = batch.to(device)
        loss = compute_loss(model, batch)

```

```

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()

    avg_loss = epoch_loss / len(loader)
    train_losses.append(avg_loss)

    # Validation loss
    model.eval()
    with torch.no_grad():
        vl = compute_loss(model, val_batch).item()
    val_losses.append(vl)

    if (epoch + 1) % 20 == 0:
        print(f'Epoch {epoch+1:3d}/{epochs}: train = {avg_loss:.4f}, val = {vl:.4f}')

print(f'\nFinal train loss: {train_losses[-1]:.4f}, val loss: {val_losses[-1]:.4f}')

```

Training for 500 epochs, 11 batches/epoch

Model: 4,841,920 parameters

```

Epoch 20/500: train = 0.3406, val = 0.3241
Epoch 40/500: train = 0.2738, val = 0.2596
Epoch 60/500: train = 0.2623, val = 0.2516
Epoch 80/500: train = 0.2428, val = 0.2280
Epoch 100/500: train = 0.2224, val = 0.2248
Epoch 120/500: train = 0.2090, val = 0.2202
Epoch 140/500: train = 0.2141, val = 0.2110
Epoch 160/500: train = 0.1976, val = 0.2048
Epoch 180/500: train = 0.1978, val = 0.1909
Epoch 200/500: train = 0.1928, val = 0.2381
Epoch 220/500: train = 0.1941, val = 0.1910
Epoch 240/500: train = 0.1880, val = 0.2007
Epoch 260/500: train = 0.1852, val = 0.1923
Epoch 280/500: train = 0.1741, val = 0.1932
Epoch 300/500: train = 0.1747, val = 0.1908
Epoch 320/500: train = 0.1684, val = 0.1814
Epoch 340/500: train = 0.1714, val = 0.1839
Epoch 360/500: train = 0.1676, val = 0.1902
Epoch 380/500: train = 0.1640, val = 0.1822
Epoch 400/500: train = 0.1634, val = 0.1914
Epoch 420/500: train = 0.1613, val = 0.1949
Epoch 440/500: train = 0.1560, val = 0.1894
Epoch 460/500: train = 0.1592, val = 0.1945

```

Epoch 480/500: train = 0.1897, val = 0.2089
Epoch 500/500: train = 0.1531, val = 0.1857

Final train loss: 0.1531, val loss: 0.1857

1.6.3 Visualize the Denoiser [2 points]

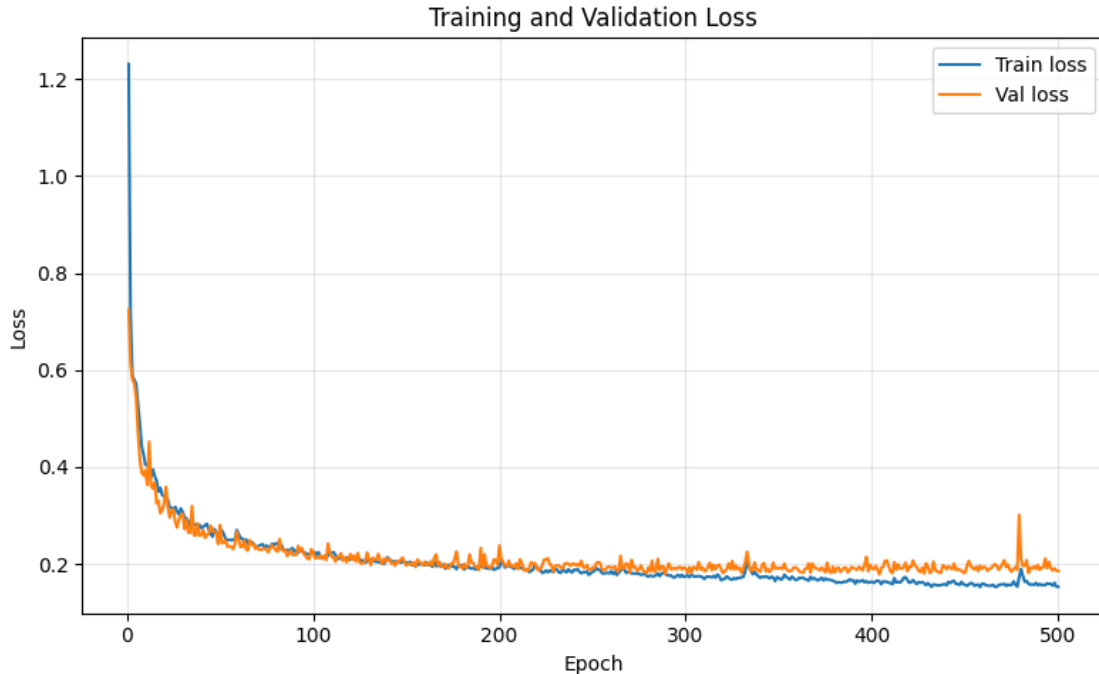
Now that the model is trained, let's examine what it has learned.

Viz 1a [2 points]: Plot the training and validation loss curves. Both should decrease and plateau. If the validation loss rises while training loss keeps dropping, the model may be memorizing the training set — it can recognize and reconstruct training images but hasn't learned generalizable structure.

```
[13]: # TODO [2 points]: Viz 1a - Loss Curve
      # Plot train_losses and val_losses on the same axes.
      # Include a legend, axis labels, and a title.

      epochs = range(1, len(train_losses) + 1)

      plt.figure(figsize=(8, 5))
      plt.plot(epochs, train_losses, label='Train loss')
      plt.plot(epochs, val_losses, label='Val loss')
      plt.xlabel('Epoch')
      plt.ylabel('Loss')
      plt.title('Training and Validation Loss')
      plt.legend()
      plt.grid(True, alpha=0.3)
      plt.tight_layout()
      plt.show()
```



Viz 1b [2 points]: Pick 3 training images. For each, show the noisy input z_t and the network's prediction $\hat{x}$ at noise levels $t = 0.1, 0.3, 0.5, 0.7, 0.9$.

Expected behavior: - At high noise (small t): $\hat{x}$ should be blurry/average-looking (the model hedges) - At low noise (large t): $\hat{x}$ should be close to the clean input (easy to denoise) - The transition from blurry to sharp should be gradual

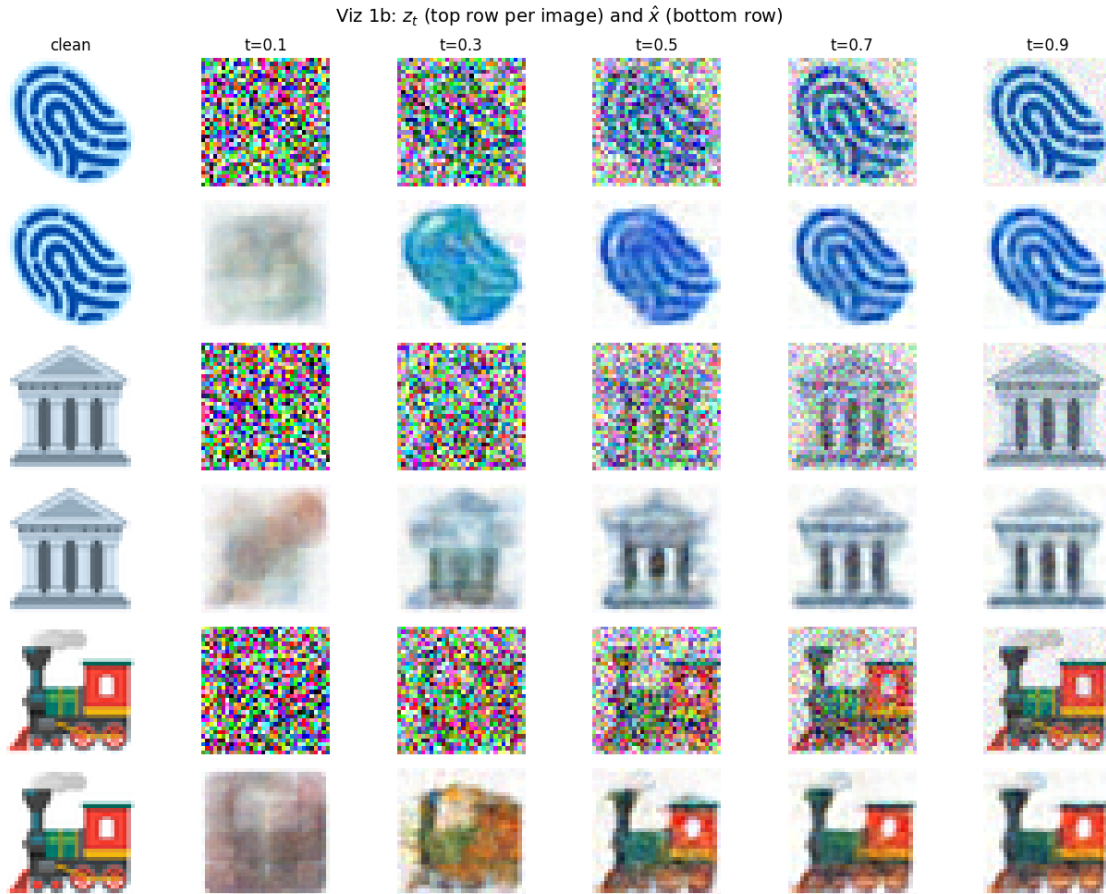
```
[14]: # TODO [2 points]: Viz 1b - Denoiser outputs at different noise levels
# For 3 training images and t_values = [0.1, 0.3, 0.5, 0.7, 0.9]:
# 1. model.eval()
# 2. For each image and each t, compute z = add_noise(x, t_val)
# 3. Get prediction: with torch.no_grad(): x_hat = model(z)
# 4. Show a grid with two rows per image: noisy input z and prediction x_hat
# Include the clean image in the first column for reference.
#
# To display: img = ((tensor[0].cpu().permute(1,2,0).numpy() + 1) / 2).clip(0, 1)

model.eval()
t_values = [0.1, 0.3, 0.5, 0.7, 0.9]
torch.manual_seed(229)
viz_indices = torch.randperm(len(images))[:3].tolist()
n_cols = 1 + len(t_values) # clean + 5 noise levels
fig, axes = plt.subplots(6, n_cols, figsize=(2.2 * n_cols, 10))
for i, idx in enumerate(viz_indices):
```

```

x = images[idx].to(device) # (3, 32, 32)
row_z = 2 * i             # noisy  $z_t$ 
row_hat = 2 * i + 1 # prediction  $\hat{x}$ 
def to_img(tensor):
    return ((tensor.cpu().permute(1, 2, 0).numpy() + 1) / 2).clip(0, 1)
clean = to_img(x)
axes[row_z, 0].imshow(clean)
axes[row_hat, 0].imshow(clean)
axes[row_z, 0].set_ylabel(f'Img {i+1}\n $z_t$ ', fontsize=9)
axes[row_hat, 0].set_ylabel(r' $\hat{x}$ ', fontsize=9)
if i == 0:
    axes[row_z, 0].set_title('clean')
for col, t_val in enumerate(t_values, start=1):
    t = torch.tensor(t_val, device=device)
    z = add_noise(x, t)
    with torch.no_grad():
        x_hat = model(z.unsqueeze(0)).squeeze(0)
    axes[row_z, col].imshow(to_img(z))
    axes[row_hat, col].imshow(to_img(x_hat))
    if i == 0:
        axes[row_z, col].set_title(f't={t_val}')
for r in (row_z, row_hat):
    for c in range(n_cols):
        axes[r, c].axis('off')
fig.suptitle('Viz 1b:  $z_t$  (top row per image) and  $\hat{x}$  (bottom row)',
             ↪ fontsize=14)
plt.tight_layout()
plt.show()

```



Viz 1c: Feed pure Gaussian noise ($t = 0$) into the model. The output $\hat{x} = f_{\theta}(\varepsilon)$ is the model's best guess of $\mathbb{E}[x | z_0]$ when given no information about which image produced the noise. Because the ViT operates on patches, the output will have visible patch structure rather than a smooth blur — each patch independently estimates the “average” patch content.

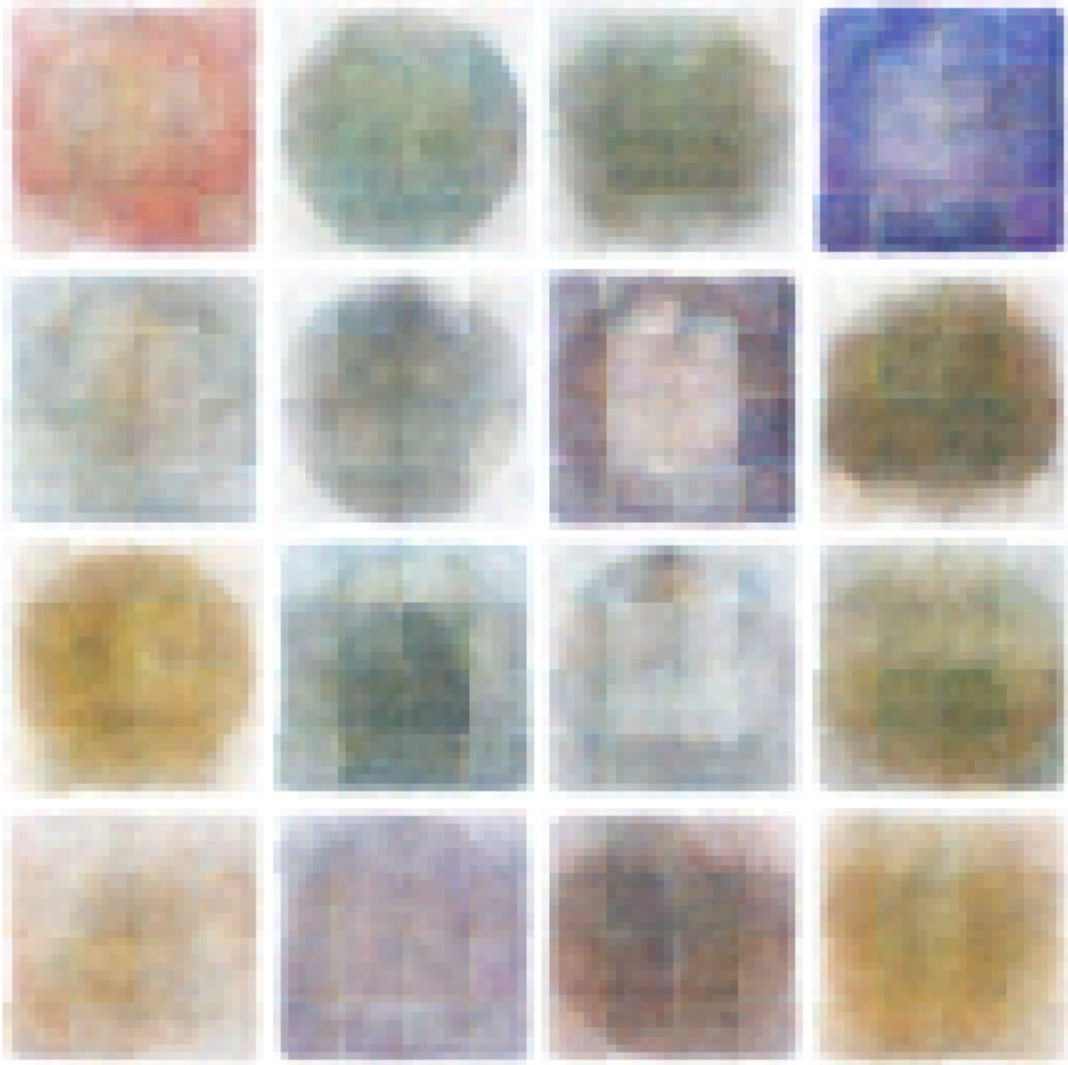
```
[15]: # TODO: Viz 1c - Denoiser on pure noise
# 1. model.eval()
# 2. Generate pure noise: torch.randn(16, 3, 32, 32, device=device)
# 3. Pass through model with torch.no_grad()
# 4. Display using show_grid()
# Viz 1c - Denoiser on pure noise
model.eval()

noise = torch.randn(16, 3, 32, 32, device=device)

with torch.no_grad():
    x_hat = model(noise)

show_grid(x_hat, nrow=4, title='Viz 1c: Model output on pure noise (t=0)')
```

Viz 1c: Model output on pure noise (t=0)



1.7 Part 2: Generate Images [6 points]

1.7.1 2a. Euler ODE Sampler [3 points]

To generate images, we integrate the learned velocity field from $t = 0$ (noise) to $t \approx 1$ (data) using Euler's method with a **linear schedule** (constant step size $\Delta t = 1/T$):

$$z_{k+1} = z_k + \frac{1}{T} \cdot v_{\theta}(z_k), \quad v_{\theta}(z_k) = \frac{\hat{x} - z_k}{1 - t_k}, \quad t_k = k/T$$

Note: while training uses logit-normal sampling to *weight* which timesteps get more gradient signal,

the sampler simply steps through time uniformly.

Implement the `sample` function below. Remember to use `model.eval()` and `@torch.no_grad()`.

```
[16]: @torch.no_grad()
def sample(model, num_samples, T=100, device='cpu'):
    """Generate images by Euler integration of the learned velocity field.

    Args:
        model: trained EmojiViT
        num_samples: number of images to generate
        T: number of Euler steps
        device: torch device

    Returns:
        (num_samples, 3, 32, 32) generated images in [-1, 1]
    """
    model.eval()

    # TODO [3 points]: Implement Euler ODE sampling.
    #
    # Step 1: Start from pure noise: z = torch.randn(num_samples, 3, 32, 32,
    ↪device=device)
    #
    # Set constant step size: dt = 1.0 / T
    #
    with torch.no_grad():
        z = torch.randn(num_samples, 3, 32, 32, device = device)
        dt = 1.0 / T
        # Step 2: Update z_t over T steps.
        for t in range(T):
            t_k = t / T
            v = (model(z) - z)/(1-t_k)
            z = z + dt * v

    # Step 3: Clamp to [-1, 1] and return
    z = torch.clamp(z, -1, 1)

    return z
    #
    pass
```

```
[17]: # Test sampler (quick check with T=10)
test_samples = sample(model, 4, T=10, device=device)
print(f'Shape: {test_samples.shape}')
print(f'Range: [{test_samples.min():.2f}, {test_samples.max():.2f}]')
assert test_samples.shape == torch.Size([4, 3, 32, 32]), 'Wrong output shape!'
print('\nSampler test passed!')
```

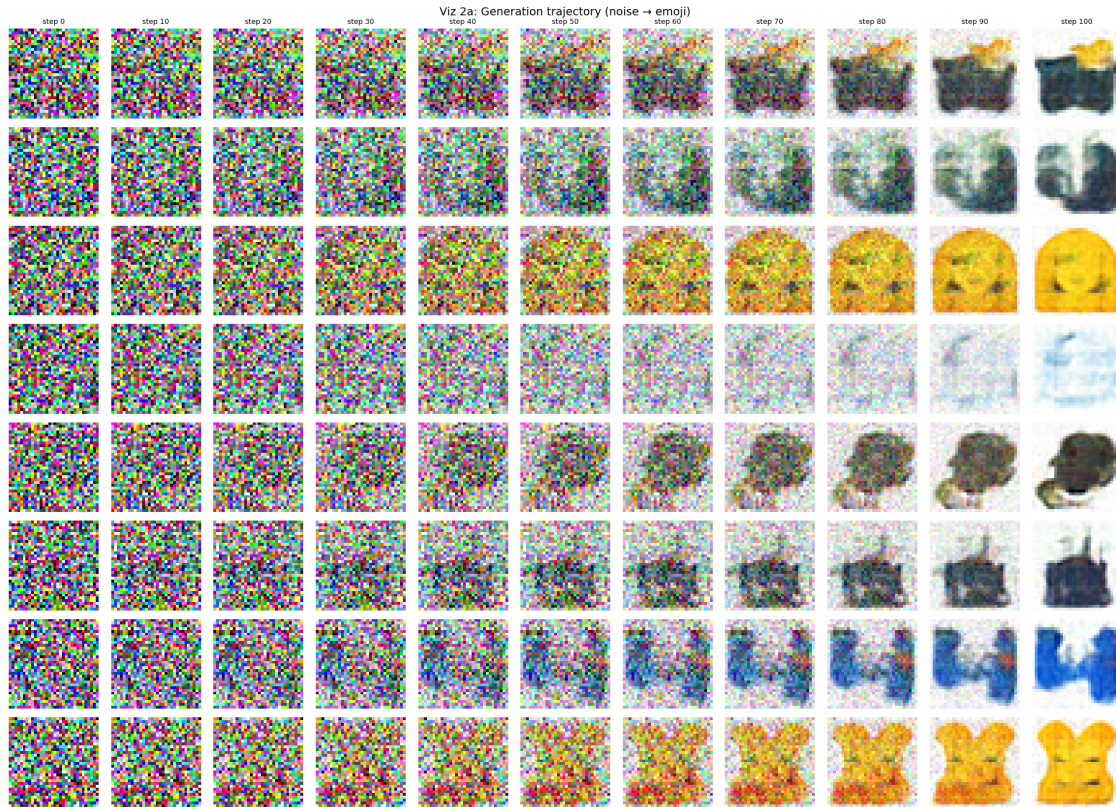
```
Shape: torch.Size([4, 3, 32, 32])
Range: [-1.00, 1.00]
```

Sampler test passed!

1.7.2 Visualize Generation [3 points]

Viz 2a [1 point]: Show the **generation trajectory** — how images emerge from noise. Generate 8 samples, saving z_k at every 10th step. Display as a grid (rows = samples, columns = timesteps). This is the reverse of Viz 0.

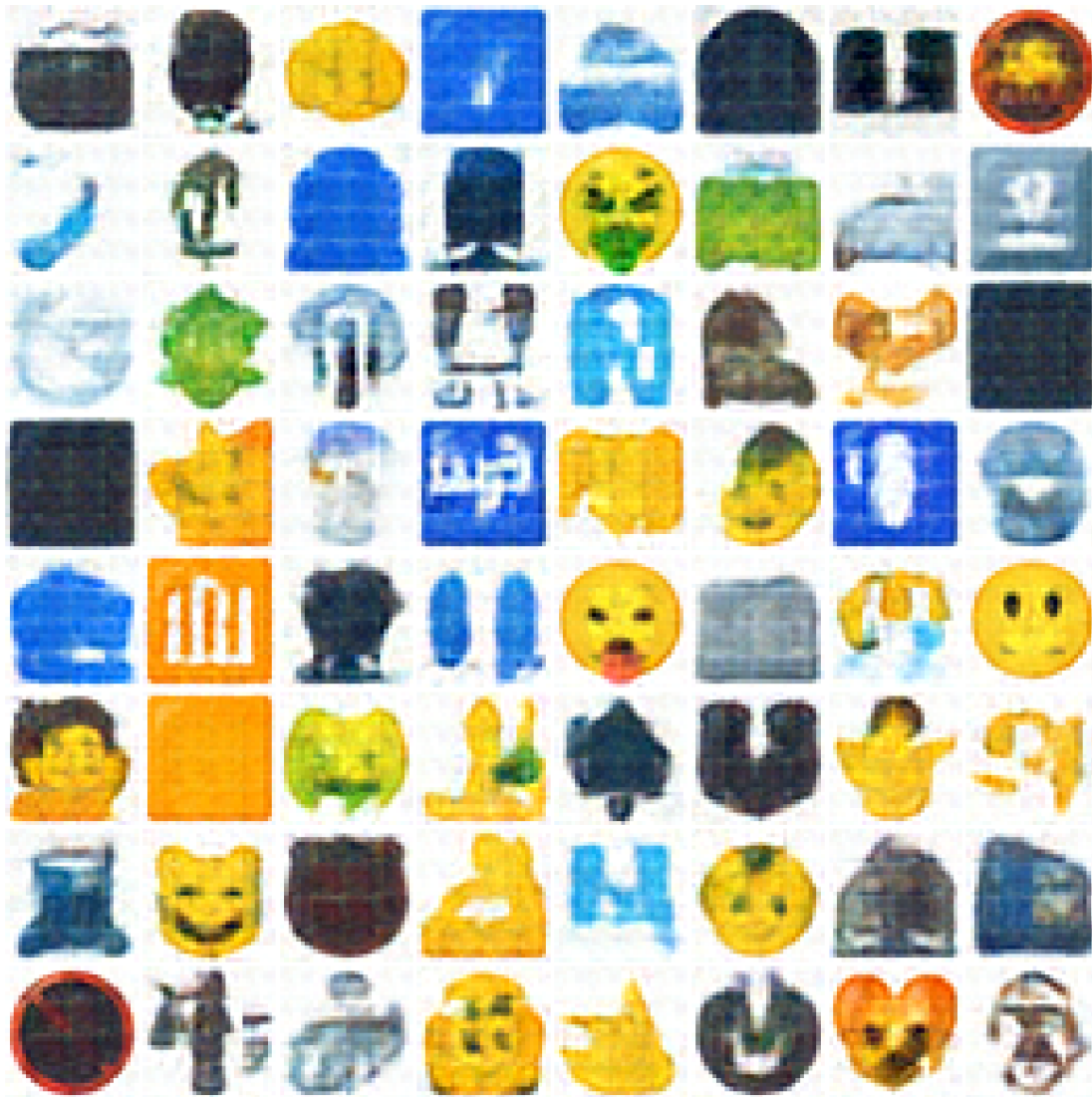
```
[18]: # TODO [1 point]: Viz 2a - Generation trajectory
# You'll need to write the sampling loop manually (not just call sample())
# so you can save intermediate states.
#
# 1. z = torch.randn(8, 3, 32, 32, device=device), dt = 1.0 / T
# 2. Save z.cpu().clone() as the first snapshot
# 3. Loop k = 0, ..., T-1: t = k/T, compute velocity, take Euler step,
#    save z every 10th step
# 4. Display as a grid (rows = samples, columns = snapshots)
model.eval()
T = 100
num_samples = 8
dt = 1.0 / T
z = torch.randn(num_samples, 3, 32, 32, device=device)
snapshots = [z.cpu().clone()]
with torch.no_grad():
    for k in range(T):
        t_k = k / T
        v = (model(z) - z) / (1 - t_k)
        z = z + dt * v
        if (k + 1) % 10 == 0:
            snapshots.append(z.cpu().clone())
# snapshots at steps 0, 10, 20, ..., 100
n_cols = len(snapshots)
fig, axes = plt.subplots(num_samples, n_cols, figsize=(1.8 * n_cols, 1.8 *
    num_samples))
def to_img(tensor):
    return ((tensor.permute(1, 2, 0).numpy() + 1) / 2).clip(0, 1)
for row in range(num_samples):
    for col, snap in enumerate(snapshots):
        axes[row, col].imshow(to_img(snap[row]))
        axes[row, col].axis('off')
    if row == 0:
        axes[row, col].set_title(f'step {col * 10}', fontsize=9)
fig.suptitle('Viz 2a: Generation trajectory (noise → emoji)', fontsize=14)
plt.tight_layout()
plt.show()
```



Viz 2b [1 point]: Generate a batch of 64 images and display as an 8x8 grid. Mine were emoji-like but still a little fuzzy and with some possible signs of memorization.

```
[19]: # TODO [1 point]: Viz 2b - Generated samples grid
# Use your sample() function to generate 64 images, then display with
↪ show_grid()
model.eval()
samples = sample(model, num_samples=64, T=100, device=device)
show_grid(samples, nrow=8, title='Viz 2b: Generated emoji (64 samples)')
```

Viz 2b: Generated emoji (64 samples)



Viz 2c [1 point]: Generate the **same** samples (same initial noise) using $T = 10, 25, 50, 100$ steps. Show side-by-side. Quality should degrade with fewer steps due to Euler discretization error, but flow matching is surprisingly good with few steps.

[20]: `# TODO [1 point]: Viz 2c - Effect of number of steps`
`# 1. Set a seed and generate initial noise: z0 = torch.randn(8, 3, 32, 32,`
`↪device=device)`
`# 2. For each T_val in [10, 25, 50, 100]:`
`# - Clone z0`
`# - dt = 1.0 / T_val`
`# - Run the Euler loop for T_val steps (t = k / T_val)`
`# - Save the result`

```

# 3. Display side-by-side (columns = T values, rows = samples)
model.eval()
torch.manual_seed(229)
z0 = torch.randn(8, 3, 32, 32, device=device)
T_values = [10, 25, 50, 100]
final_states = []
with torch.no_grad():
    for T_val in T_values:
        z = z0.clone()
        dt = 1.0 / T_val
        for k in range(T_val):
            t_k = k / T_val
            v = (model(z) - z) / (1 - t_k)
            z = z + dt * v
        final_states.append(z.cpu().clamp(-1, 1))
fig, axes = plt.subplots(8, 4, figsize=(10, 16))
def to_img(tensor):
    return ((tensor.permute(1, 2, 0).numpy() + 1) / 2).clip(0, 1)
for col, T_val in enumerate(T_values):
    for row in range(8):
        axes[row, col].imshow(to_img(final_states[col][row]))
        axes[row, col].axis('off')
    if row == 0:
        axes[row, col].set_title(f'T = {T_val}', fontsize=12)
fig.suptitle('Viz 2c: Same initial noise, different step counts', fontsize=14)
plt.tight_layout()
plt.show()

```

Viz 2c: Same initial noise, different step counts



1.8 Extra Credit

Extra credit is submitted separately on Canvas using the **HW 3 (EC)** assignment. Submit a PDF (made with LaTeX) outlining the problem, your approach, results (including tables or figures), and a discussion. Also submit your code (.py or .ipynb). Graded 1-5 points based on quality, though could be higher for novel research directions. Feel free to pitch directions!

1.8.1 EC1: Time Conditioning Comparison

Add a time-conditioned version of the model (e.g., FiLM or concatenation). Train both versions and compare loss curves, sample quality, and denoiser visualizations. When does time conditioning help? Connect to [Sahraee-Ardakan et al. \(2026\)](#).

1.8.2 EC2: Prediction Space Ablation

Implement ε -prediction and/or v -prediction and compare to x -prediction. Replicate the ablation from [Li & He \(2025\)](#), Table 2. Key question: does ε -prediction fail more with no time conditioning?

1.8.3 EC3: Heun’s Method (2nd Order Sampler)

Implement Heun’s method for ODE sampling. Compare to Euler at the same number of function evaluations (Heun with N steps vs. Euler with $2N$ steps). Connect to [Karras et al. \(2022\)](#).

1.8.4 EC4: Noise Schedule Ablation

The assignment uses logit-normal time sampling ($\mu = -0.8, \sigma = 0.8$) following [Li & He \(2025\)](#). Compare against: - **Uniform**: $k \sim \text{Uniform}\{0, \dots, T-1\}$ - **Different logit-normal parameters**: vary μ and σ - Look for one other alternative sampler. Karras mentions a few, or the “Curriculum Sampling” paper ([arXiv:2603.12517](#)) argues for a two-phase schedule

Compare at least three strategies, reporting loss curves and sample quality

1.8.5 EC5: Text Conditioning

Add a pre-trained language model (e.g., CLIP) to condition on emoji text descriptions. Can you generate custom emoji from new descriptions?

1.8.6 EC6: Bells and Whistles

Make the model generate better emoji. Pick a quantitative metric (e.g., FID — does it work well at this scale?) and systematically test what helps and what hurts. Possible knobs to turn: learning rate, optimizer momentum, EMA of model weights, learning rate schedulers, data augmentation, model size, training duration, weight decay, early stopping. Report your metric for each ablation in a table. If you get an “autoresearch” loop working on this, I’d love to hear about how you set it up and how it did!

1.8.7 EC7: Memorization or Generalization?

Is the model memorizing training images, or learning to generate novel emoji? Design and run an experiment to answer this. Include a short literature review (2–4 papers) on how memorization in generative models is detected and measured, and justify why your chosen method is appropriate for this setting. Some possible approaches (you are not limited to these): - Nearest-neighbor analysis between generated and training images - Comparing generation diversity to training set diversity - Training on a subset and checking if generations resemble held-out images - Interpolation in noise space

1.9 Submission

Export this notebook to PDF and submit on Canvas.

```
!jupyter nbconvert --to pdf --output=yourname_hw3.pdf hw3.ipynb
```